Software Requirements Specification (SRS)

CFDI and Bank Statement Reconciliation Platform – MVP core

2026-10-28

1. Introduction

1.1 Purpose

This document specifies the behaviour of the MVP described in the Business Requirements Specification (BRS v0.7). It is generated from the OpenSpec change mvp-reconciliation-core (specs, design and tasks), which remains the source of truth: when the two differ, the OpenSpec files win and this document is regenerated.

1.2 Scope

A multi-company backend that ingests CFDI 4.0 XML and CSV statements in a documented reference format, proposes links between movements and invoices, lets users review doubtful links, and records every decision in a tamper-evident audit log. It is a portfolio demonstration running on anonymised or synthetic data; its results have no fiscal or legal validity.

1.3 Document conventions

Requirements use SHALL for mandatory behaviour. Each requirement has one or more scenarios in WHEN / THEN form; each scenario is a candidate test case. Choices made by the author and not yet confirmed by the project owner are marked (my call) in the design section.

1.4 Definitions

Term	Meaning
CFDI	Comprobante Fiscal Digital por Internet, the Mexican electronic invoice (XML)
Payment complement	CFDI of type P that relates a payment to one or more invoices
Credit note	CFDI of type E that reduces an invoice's payable amount
UUID	Unique identifier of a CFDI stamped by the SAT
SAT	Mexican tax authority
PUE / PPD	Payment in a single instalment / in instalments
EFOS	Companies the SAT lists as issuers of simulated operations
Movement	One line of a bank statement
Link	Association between a movement and a CFDI with an applied amount
Reconciled / suggested / pending	Item states derived from links; suggested items await human review
Direction	Issued when the company is the CFDI issuer, received when it is the receiver
Complement due date	Fifth calendar day of the month after a payment on a PPD invoice
Global invoice	CFDI to the general public (generic RFC) that covers many operations
Platform operator	Role outside any company that creates companies

2. Overall description

2.1 Users and roles

Role	Can do
Administrator	Manage users and settings of a company; re-run failed jobs
Accountant	Import files, review and decide on links, classify movements
Viewer / auditor	Read data and the audit log, no changes
System	Create deterministic links, run jobs and status checks
Platform operator	Create companies with their RFC and first administrator

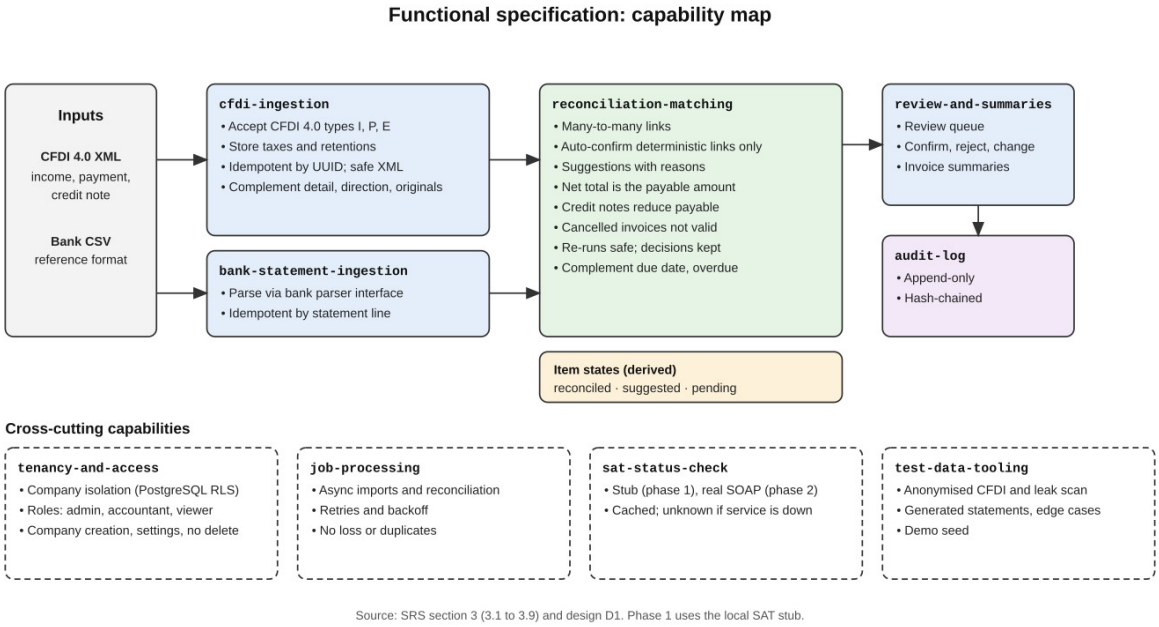
2.2 Assumptions and constraints

- The MVP is delivered in 1–2 weeks and runs locally with Docker Compose.
- Phase 1 checks CFDI status against a local stub; phase 2 connects to the real SAT service after one real payment CFDI has been tested.
- Real CFDI never enter the repository; only anonymised files do.
- Amounts are exact to two decimals.
- Original CFDI and statement files are kept for at least five years and are never deleted through the API.
- Only credit movements and issued income CFDI are matched in this version; the data model keeps the direction so received invoices can be added later.
- Several fiscal rules come from secondary sources and are listed as open questions in section 4.5; they are isolated in settings or warnings.

2.3 Traceability to business requirements

BRS requirement	Covered by
BR-01, BR-11	CFDI Ingestion
BR-02, BR-15	Bank Statement Ingestion
BR-03, BR-04, BR-05, BR-14	Reconciliation Matching
BR-06, BR-12, BR-26	Review and Summaries
BR-07	Audit Log
BR-08	Tenancy and Access
BR-10, BR-29, BR-30, BR-31	SAT Status Check
BR-23	Job Processing
BR-17	Test Data Tooling
BR-18, BR-19, BR-20, BR-21, BR-22, BR-24, BR-25, BR-27, BR-28	Design (section 4), tasks and ADRs
BR-09, BR-13, BR-16	Deferred (stretch goals)

3. Functional specification



Functional specification at a glance (summary of section 3).

Each subsection is one capability. User stories appear in the purpose of each capability.

3.1 CFDI Ingestion

cfdi-ingestion

Purpose and user story. Lets an accountant load CFDI 4.0 XML files (income invoices, payment complements, credit notes) into a company's workspace so they can be reconciled. User story: as an accountant, I want to upload invoices for a company so that I can start reconciling.

Requirement: Accept CFDI 4.0 XML files

The system SHALL accept CFDI 4.0 XML files of type income (I), payment (P) and credit note / egress (E) uploaded to a company workspace, and SHALL store for each: UUID, type, issuer RFC, receiver RFC, issue date, currency, total, payment method (PUE/PPD) and, where present, the related invoice UUIDs with amounts.

Scenario: Income invoice accepted

- **WHEN** a user uploads a valid CFDI 4.0 income XML to a company
- **THEN** the system stores it with its UUID, RFCs, date, currency and total and reports it as imported

Scenario: Payment complement accepted

- **WHEN** a user uploads a valid CFDI 4.0 payment complement 2.0 that relates invoice UUID A for 5,800.00
- **THEN** the system stores the payment CFDI together with the relation to UUID A and the related amount

Scenario: Unsupported version or type rejected

- **WHEN** a user uploads a CFDI of a version other than 4.0, or a file that is not a CFDI
- **THEN** the system rejects that file with a reason and imports nothing from it

Requirement: Taxes, discounts and retentions are stored

The system SHALL store, for each income CFDI, the subtotal, the discount, the total transferred taxes, the total retained taxes, any local taxes transferred or retained, and the total, and SHALL reject a CFDI whose total differs from subtotal minus discount plus transferred taxes minus retained taxes (federal and local in both cases), or whose discount exceeds the subtotal. The system SHALL check only this arithmetic on the declared header amounts and SHALL NOT recompute taxes from rates.

Scenario: Invoice with retention

- **WHEN** a user uploads an income CFDI with subtotal 1,690.00, discount 0.00, transferred tax 270.40, retained tax 21.13 and total 1,939.27
- **THEN** the system stores all amounts and imports the file

Scenario: Invoice with discount

- **WHEN** a user uploads an income CFDI with subtotal 1,000.00, discount 100.00, transferred tax 144.00 and total 1,044.00
- **THEN** the system stores all amounts and imports the file

Scenario: Inconsistent total

- **WHEN** a CFDI declares a total that does not equal subtotal minus discount plus transferred taxes minus retained taxes
- **THEN** the system rejects the file with the reason

Scenario: Discount above subtotal

- **WHEN** a CFDI declares a discount greater than its subtotal
- **THEN** the system rejects the file with the reason

Requirement: Idempotent by UUID

The system SHALL treat a CFDI as already imported when a CFDI with the same UUID exists in the same company, and SHALL NOT create a duplicate or alter the stored data.

Scenario: Same file uploaded twice

- **WHEN** the same CFDI file is uploaded twice to one company
- **THEN** the second upload creates no new record and the import report lists it as already imported

Scenario: Same UUID in two companies

- **WHEN** a CFDI with the same UUID is uploaded to two different companies the user can access
- **THEN** each company holds its own isolated copy

Requirement: Safe handling of untrusted XML

The system SHALL parse uploaded XML without resolving external entities, external DTDs or network references, and SHALL enforce a maximum file size and a maximum number of files per upload.

Scenario: External entity attempt

- **WHEN** a user uploads an XML file that declares an external entity pointing to a local file or a network address
- **THEN** the system rejects the file, does not read the referenced resource and records the rejection

Scenario: Oversized upload

- **WHEN** an upload exceeds the configured size or file-count limit
- **THEN** the system rejects the upload with a clear reason and imports nothing

Requirement: Per-file import report

The system SHALL return, for each upload, a report with one result per file: imported, already imported, or rejected with reason; one invalid file SHALL NOT prevent valid files in the same upload from being imported.

Scenario: Mixed upload

- **WHEN** a user uploads three files of which one is invalid and one was imported before
- **THEN** the report shows one imported, one already imported and one rejected, and only the valid new file is stored

Requirement: Credit notes linked to invoices

The system SHALL record the relation between a credit note and the invoice it refers to when the CFDI declares one.

Scenario: Credit note relation stored

- **WHEN** a credit note CFDI declares a related invoice UUID
- **THEN** the system stores the relation so that the matching capability can reduce the invoice's payable amount

Requirement: CFDI must involve the company

The system SHALL accept a CFDI into a company only when the issuer or the receiver RFC equals the company's RFC, and SHALL record its direction: issued when the company is the issuer, received when it is the receiver.

Scenario: Issued invoice

- **WHEN** a user uploads to a company with RFC AAA010101AAA a CFDI whose issuer RFC is AAA010101AAA
- **THEN** the system stores it with direction issued

Scenario: Unrelated CFDI

- **WHEN** a user uploads a CFDI in which neither RFC is the company's RFC

- **THEN** the system rejects the file with the reason "does not involve this company"

Requirement: Payment complement detail

The system SHALL accept only payment complements of version 2.0 and SHALL store each payment (date, form of payment, currency, exchange rate, amount, operation number when present) and, for each payment, each related invoice with its installment number, previous balance, amount paid, remaining balance and currency. The system SHALL check that the remaining balance equals the previous balance minus the amount paid.

Scenario: Complement with consistent balances

- **WHEN** a user uploads a payment complement that relates invoice A with previous balance 11,600.00, amount paid 5,800.00 and remaining balance 5,800.00
- **THEN** the system stores the payment and the related invoice line

Scenario: Inconsistent balance

- **WHEN** a payment complement declares previous balance 11,600.00, amount paid 5,800.00 and remaining balance 4,000.00
- **THEN** the system imports the complement, flags it for review with the reason "inconsistent balance" and does not use it for automatic confirmation

Scenario: Old complement version

- **WHEN** a user uploads a payment complement of a version other than 2.0
- **THEN** the system rejects the file with the reason

Requirement: Relations between CFDI keep the SAT code

The system SHALL store each relation between CFDI with its SAT relation code (including credit note 01, return 03, substitution 04 and advance application 07), and SHALL store relations of unsupported codes without applying them to payable amounts.

Scenario: Substitution recorded

- **WHEN** an income CFDI declares relation code 04 to a previous UUID
- **THEN** the system stores the relation as a substitution of that CFDI

Requirement: Global invoices and generic RFCs

The system SHALL mark a CFDI as global when it carries the global-information node or its receiver RFC is the generic national RFC (XAXX010101000), and SHALL accept the generic foreign RFC (XEXX010101000) as a receiver.

Scenario: Global invoice

- **WHEN** a user uploads an income CFDI with receiver RFC XAXX010101000 and global information
- **THEN** the system stores it marked as global

Requirement: Dates as declared

The system SHALL store the issue date and time exactly as declared (local time, no time zone) and the stamp date and time separately.

Scenario: Issue date near midnight

- **WHEN** a CFDI is issued at 2026-09-30T23:50:00 and stamped at 2026-10-01T00:05:00
- **THEN** the system stores both values unchanged and the issue date remains 2026-09-30

Requirement: Original files retained

The system SHALL keep every accepted original XML file unchanged together with its hash, SHALL NOT delete it through any API, and SHALL keep it for a configurable retention period of at least five years.

Scenario: Original available

- **WHEN** a user requests the original file of an imported CFDI
- **THEN** the system returns the stored XML byte for byte

3.2 Bank Statement Ingestion

bank-statement-ingestion

Purpose and user story. Lets an accountant load a bank's CSV statement into a company's workspace so its movements can be reconciled. User story: as an accountant, I want to upload a bank statement so that its movements become candidates for matching.

Requirement: Accept CSV statements in the reference format

The system SHALL accept CSV statements in a documented reference format with the columns fecha, concepto, abono, cargo, saldo (header row required, ISO dates, decimal point, no thousands separator) and SHALL store each line as a movement with date, description, credit amount (abono), debit amount (cargo) and balance. The reference format SHALL be identified by the bank identifier reference, and the format of a real bank is a later, additional parser.

Scenario: Valid statement imported

- **WHEN** a user uploads a valid statement containing two credit lines
- **THEN** the system stores two movements of type credit with their dates, descriptions, amounts and balances

Scenario: Malformed line

- **WHEN** a statement contains a line with an unparseable date or amount
- **THEN** the system rejects the whole statement with the line number and reason, and imports no movements from it

Requirement: Extensible parser per bank

The system SHALL select the parser by an explicit bank identifier supplied with the upload, and adding support for another bank SHALL NOT require changes to matching, review or audit behaviour.

Scenario: Unknown bank identifier

- **WHEN** a user uploads a statement with a bank identifier that has no parser
- **THEN** the system rejects the upload and lists the supported banks (initially only reference)

Requirement: Idempotent by statement line

The system SHALL treat a movement as already imported when a movement with the same bank account, date, description, amount and running balance exists in the same company, and SHALL NOT create a duplicate.

Scenario: Overlapping statements

- **WHEN** a user uploads two statements whose date ranges overlap
- **THEN** the overlapping movements are stored once and the report states how many were new and how many were already imported

Scenario: Two legitimate identical movements

- **WHEN** a statement contains two lines with the same date, description and amount but different running balances
- **THEN** both are stored as separate movements

Requirement: Amounts handled exactly

The system SHALL represent monetary amounts without floating-point rounding errors and SHALL preserve two decimal places of the source.

Scenario: Cent-exact sums

- **WHEN** movements of 0.10 and 0.20 are summed
- **THEN** the result is exactly 0.30

Requirement: Reference extracted from the description

The system SHALL expose for each movement an optional reference (tracking key or reference number) extracted by the bank parser from the description, using the token RASTREO:<key> or REF:<number> in the reference format.

Scenario: Tracking key present

- **WHEN** a statement line has the description "SPEI REC 0034 RASTREO:ABC123 PAGO FACT 0001"
- **THEN** the movement stores the reference ABC123

Scenario: No reference

- **WHEN** a statement line has no reference token
- **THEN** the movement stores no reference and the import is not rejected

Requirement: List of supported banks

The system SHALL let a user list the supported bank identifiers.

Scenario: Supported banks listed

- **WHEN** a user asks for the supported banks
- **THEN** the system returns the identifiers that have a parser (initially only reference)

Requirement: Original statements retained

The system SHALL keep every accepted statement file unchanged with its hash, SHALL NOT delete it through any API, and SHALL keep it for a configurable retention period of at least five years.

Scenario: Original statement available

- **WHEN** a user requests the original file of an imported statement
- **THEN** the system returns the stored file byte for byte

3.3 Reconciliation Matching

reconciliation-matching

Purpose and user story. Links bank movements to CFDI according to the business rules in the BRS, and classifies every item so that clear cases are automated and doubtful ones go to a human. User story: as an accountant, I want obvious matches made for me so that I only spend time on exceptions.

Requirement: Many-to-many links between movements and CFDI

The system SHALL allow one movement to be linked to several CFDI and one CFDI to be linked to several movements, recording for each link the amount applied.

Scenario: Partial payments on one invoice

- **WHEN** two movements of 5,800.00 are linked to an invoice of 11,600.00
- **THEN** the invoice shows 11,600.00 applied in total and no amount pending

Scenario: One payment covering two invoices

- **WHEN** one movement of 10,000.00 is linked to invoice A for 6,000.00 and invoice B for 4,000.00
- **THEN** the movement shows 10,000.00 fully applied and each invoice shows its own applied amount

Requirement: Automatic confirmation only for deterministic links

The system SHALL mark a link as reconciled automatically only when it is deterministic: a payment complement references the invoice UUID, its payment amount equals the movement amount, and the movement date is within the configured tolerance of the payment date. All other candidate links SHALL be proposed as suggested.

Scenario: Payment complement matches movement

- **WHEN** a payment complement relates invoice A for 5,800.00 on 2026-10-10 and a credit movement of 5,800.00 exists dated 2026-10-10
- **THEN** the system links the movement to the payment CFDI and invoice A and marks the link reconciled

Scenario: Movement without complement

- **WHEN** a credit movement of 5,800.00 dated 2026-10-20 has no payment complement but its description and amount point to invoice A
- **THEN** the system proposes a link to invoice A marked suggested, with the reasons, and does not mark it reconciled

Requirement: Confidence and explanation for suggestions

The system SHALL attach to each suggested link a confidence level and the list of criteria that matched (amount, date proximity, RFC, reference text), and SHALL be able to show why a link was proposed.

Scenario: Reasons visible

- **WHEN** a user opens a suggested link
- **THEN** the system lists the matched criteria and the confidence level

Requirement: Payable amount is the net total

The system SHALL use the CFDI total (subtotal minus discount plus transferred taxes minus retained taxes) as the amount a payer is expected to settle, not the subtotal.

Scenario: Retention reduces the payment

- **WHEN** an invoice has subtotal 1,690.00, tax 270.40, retention 21.13 and a credit movement of 1,939.27 exists with a matching reference
- **THEN** the system treats the invoice as fully covered by that movement and shows no amount pending

Requirement: Item states

The system SHALL assign every movement and every invoice exactly one state: reconciled, suggested (to review) or pending, derived from its links and the amounts applied.

Scenario: Invoice partially paid

- **WHEN** an invoice has 5,800.00 applied through a reconciled link out of 11,600.00 and an additional suggested link of 5,800.00
- **THEN** the invoice state is suggested and its summary shows 50% paid and 5,800.00 pending

Scenario: Movement with no candidate

- **WHEN** no CFDI satisfies any criterion for a movement
- **THEN** the movement state is pending

Requirement: Credit notes reduce the payable amount

The system SHALL subtract the total of related credit notes from the amount payable of an invoice before computing paid percentage and pending amount.

Scenario: Credit note applied

- **WHEN** an invoice of 11,600.00 has a related credit note of 1,160.00 and 10,440.00 is paid and reconciled

- **THEN** the invoice shows no amount pending

Requirement: Cancelled documents are not reconciled as valid

The system SHALL NOT propose or confirm links that depend on a CFDI whose SAT status is cancelled, SHALL treat a cancellation that is still in process as not cancelled, and SHALL move to review the existing links that depend on an invoice, payment complement or credit note later found cancelled.

Scenario: Invoice found cancelled

- **WHEN** the status check reports that an invoice with a reconciled link is cancelled
- **THEN** the link is moved to review with the reason "cancelled invoice" and the change is recorded in the audit log

Scenario: Payment complement found cancelled

- **WHEN** the status check reports that a payment complement used by a reconciled link is cancelled
- **THEN** the link is moved to review with the reason "cancelled payment complement" and the change is recorded in the audit log

Scenario: Cancellation in process

- **WHEN** the status check reports that the cancellation of an invoice is in process
- **THEN** the invoice keeps its links and shows the cancellation status, and no link is moved to review

Scenario: Replacement found

- **WHEN** an invoice is found cancelled and an imported CFDI declares relation code 04 to it
- **THEN** the system proposes the replacement CFDI as a suggested link for the affected movements

Requirement: Movements without CFDI

The system SHALL allow a movement to be classified as having no CFDI with a reason (loan, partner contribution, tax payment, late invoicing, other) and SHALL exclude such movements from the pending list.

Scenario: Loan deposit classified

- **WHEN** a user classifies a deposit as a loan
- **THEN** the movement leaves the pending list and shows the reason

Requirement: Re-running reconciliation is safe

The system SHALL produce the same links and states when reconciliation is run again over unchanged data, and SHALL NOT overwrite a decision made by a user.

Scenario: Second run

- **WHEN** reconciliation runs twice with no new data
- **THEN** the second run creates no new links and changes no states

Scenario: User decision preserved

- **WHEN** a user has rejected a suggested link and reconciliation runs again
- **THEN** the rejected link is not proposed again unless the user reopens it

Requirement: Direction of matching

The system SHALL propose links only between credit movements and income CFDI issued by the company, and SHALL leave received CFDI and debit movements unmatched in this version, keeping the direction of each CFDI so that matching them can be added later.

Scenario: Debit movement

- **WHEN** a debit movement of 5,800.00 exists and an issued invoice of 5,800.00 exists
- **THEN** the system does not propose a link between them

Requirement: Payer did not withhold

The system SHALL propose, marked suggested and never confirmed automatically, a link when a credit movement equals the CFDI total plus its retained taxes, and SHALL show that the payment appears not to include the retention.

Scenario: Gross payment

- **WHEN** an invoice has total 1,939.27 with retained tax 21.13 and a credit movement of 1,960.40 exists with a matching reference
- **THEN** the system proposes the link with the criterion "amount equals total before retentions" and the reason is visible

Requirement: Reference match

The system SHALL treat a movement reference equal to a payment's operation number as a matching criterion that raises confidence.

Scenario: Tracking key equals operation number

- **WHEN** a movement reference equals the operation number of a payment in a complement and the amounts are equal
- **THEN** the criterion "reference equals operation number" appears among the reasons

Requirement: Foreign-currency payments

The system SHALL record on each link the amount applied in the invoice currency, the movement amount in the movement currency and the exchange rate declared in the payment complement, and SHALL NOT compute exchange gains or losses.

Scenario: Dollar invoice paid in pesos

- **WHEN** an invoice of 1,000.00 USD has a complement payment of 17,500.00 MXN at the declared rate 17.50 and a credit movement of 17,500.00 MXN exists
- **THEN** the link applies 1,000.00 USD to the invoice and shows 17,500.00 MXN for the movement with the declared rate

Requirement: Payment complement due date

The system SHALL compute for each payment received on a PPD invoice a complement due date equal to the fifth calendar day of the month following the payment date, SHALL show an invoice as complement overdue when that date has passed without a complement for the payment, and SHALL allow a company setting, off by default, that moves the due date to the next business day.

Scenario: Complement overdue

- **WHEN** a movement of 5,800.00 paid on 2026-10-10 against a PPD invoice has no complement on 2026-11-06
- **THEN** the invoice shows complement overdue with due date 2026-11-05

Scenario: Due date on a weekend

- **WHEN** the fifth day falls on a Saturday and the company setting is off
- **THEN** the due date stays on that Saturday

Requirement: Applied amount never exceeds the payable amount

The system SHALL flag for review, with the reason "overpaid", any invoice whose reconciled applied amounts exceed its payable amount.

Scenario: Overpayment

- **WHEN** reconciled links apply 12,000.00 to an invoice whose payable amount is 11,600.00
- **THEN** the invoice is flagged for review with the reason "overpaid"

Requirement: Global invoices are not matched by RFC

The system SHALL NOT use RFC criteria to propose links for global invoices and SHALL show them as pending with the reason "global invoice".

Scenario: Global invoice pending

- **WHEN** a global invoice exists and a credit movement mentions the generic RFC
- **THEN** no link is proposed and the invoice shows pending with the reason "global invoice"

3.4 Review and Summaries

review-and-summaries

Purpose and user story. Gives users a review queue for doubtful matches, the ability to decide on them, and clear paid / pending summaries, through a documented API and a minimal UI. User stories: as an accountant, I want a queue of doubtful matches with their reasons so that I can decide quickly; as a finance lead, I want to see what is paid, partially paid and pending so that I can follow up.

Requirement: Review queue

The system SHALL list, per company, all suggested links and flagged items awaiting a decision, ordered by confidence level and then date, with the reasons for each.

Scenario: Queue shows suggestions

- **WHEN** a user opens the review queue of a company with two suggested links
- **THEN** both are listed with their criteria and confidence, and reconciled items are not listed

Requirement: Decisions on suggestions

The system SHALL let an authorised user confirm, reject, or change (different CFDI or different applied amount) a suggested link, and SHALL record the decision with user and timestamp.

Scenario: Confirm suggestion

- **WHEN** a user confirms a suggested link
- **THEN** the link becomes reconciled and the item states are recomputed

Scenario: Reject suggestion

- **WHEN** a user rejects a suggested link
- **THEN** the link is removed from the queue, is not proposed again, and the movement returns to pending if it has no other links

Scenario: Change suggestion

- **WHEN** a user re-links a suggested movement to a different invoice
- **THEN** the old suggestion is closed, a new reconciled link is created by the user, and both actions are recorded

Requirement: Per-invoice summary

The system SHALL show for each invoice its total, credit notes applied, amount paid, percentage paid, amount pending, state, and the linked movements and payment CFDI.

Scenario: Summary of the worked example

- **WHEN** invoice A of 11,600.00 has a reconciled link of 5,800.00 and a suggested link of 5,800.00
- **THEN** the summary shows 50% paid, 5,800.00 pending, both movements, and the payment CFDI for the reconciled one

Requirement: Company overview

The system SHALL show per company the counts and amounts of reconciled, suggested and pending invoices and movements.

Scenario: Overview totals

- **WHEN** a user opens the overview of a company
- **THEN** counts and amounts per state are displayed and add up to the totals of imported invoices and movements

Requirement: Documented, versioned API

The system SHALL expose all user-facing capabilities through a versioned HTTP API described by a machine-readable OpenAPI document, and the minimal UI SHALL use only that API.

Scenario: API document available

- **WHEN** a developer requests the API description
- **THEN** the system returns an OpenAPI document that covers every endpoint of the current version

Requirement: Clear demo disclaimer

The system SHALL state in the UI and API documentation that results have no fiscal or legal validity.

Scenario: Disclaimer visible

- **WHEN** a user opens the overview or the API description
- **THEN** the disclaimer is shown

Requirement: Complement status in the summary

The system SHALL show on each invoice summary its direction, its payment method, the complement status of each payment (received, missing, overdue) and the due date of missing complements.

Scenario: Missing complement

- **WHEN** a PPD invoice has a reconciled payment with no complement yet
- **THEN** the summary shows the payment with complement missing and its due date

Requirement: Filter invoices by complement status

The system SHALL let users list invoices filtered by complement status (missing, overdue), so that overdue complements can be followed up.

Scenario: Overdue list

- **WHEN** a user filters invoices by complement overdue
- **THEN** only invoices with at least one overdue complement are listed with their due dates

Requirement: Overview alerts

The system SHALL show in the company overview the number of invoices with complement overdue, cancellation in process, EFOS warning, or marked global.

Scenario: Alert counts

- **WHEN** a company has two invoices with complement overdue and one with an EFOS warning
- **THEN** the overview shows 2 and 1 for those alerts

Requirement: Cash payment warning

The system SHALL show a non-blocking warning on a payment whose form of payment is cash and whose amount exceeds 2,000.00, stating that results have no fiscal validity and that deductibility must be checked.

Scenario: Cash payment above the limit

- **WHEN** a payment complement declares form of payment 01 for 3,000.00
- **THEN** the payment is shown with a warning and its state is unchanged

Requirement: One decision resource

The system SHALL expose confirm, reject and change as decisions recorded on a link, each decision creating one audit entry.

Scenario: Reject recorded

- **WHEN** a user rejects a suggested link
- **THEN** one decision of type reject is recorded with the user and one audit entry is added

3.5 Audit Log

audit-log

Purpose and user story. Keeps a complete, append-only and tamper-evident record of what happened to a company's data and who decided what, so that every reconciliation result can be explained afterwards. User story: as an auditor, I want to see who matched what and when so that I can trust the result.

Requirement: Record relevant actions

The system SHALL record an audit entry for every import (accepted, duplicate, rejected), every automatic link, every user decision (confirm, reject, change, classify), every status change caused by a SAT check, and every role or access change, each with actor (user or system), company, timestamp, action and affected items.

Scenario: User confirms a link

- **WHEN** a user confirms a suggested link
- **THEN** an entry is added with the user, the time, the action, the movement and the CFDI involved

Scenario: Automatic link

- **WHEN** the system creates a reconciled link automatically
- **THEN** an entry is added with the system as actor and the criteria that justified it

Requirement: Append-only

The system SHALL NOT allow audit entries to be modified or deleted through any API or application role.

Scenario: Attempt to modify

- **WHEN** any application role attempts to update or delete an audit entry
- **THEN** the operation is refused and the attempt itself is recorded

Requirement: Tamper evidence

The system SHALL chain entries so that altering or removing an entry can be detected, and SHALL provide a verification operation that reports the first inconsistent entry, if any.

Scenario: Intact log

- **WHEN** an auditor runs the verification on an untouched log
- **THEN** the system reports the log as consistent

Scenario: Altered entry

- **WHEN** an entry is altered directly in storage and the verification runs
- **THEN** the system reports the position of the first inconsistent entry

Requirement: Read access for auditors

The system SHALL let users with the viewer or auditor role read and filter the audit log of their companies by date, actor, action and item, without allowing any change.

Scenario: Filter by item

- **WHEN** an auditor filters the log by a CFDI UUID
- **THEN** only entries involving that CFDI are returned, in chronological order

Requirement: Company isolation of the log

The system SHALL show a user only the audit entries of companies the user can access.

Scenario: Other company's entries

- **WHEN** a user queries the log of a company they cannot access
- **THEN** the system returns no entries and reveals nothing about that company

3.6 Tenancy and Access

tenancy-and-access

Purpose and user story. Lets several companies, and accounting firms serving many of them, use one installation with strict data isolation enforced below the application layer, and with roles that limit what each user can do. User story: as a firm, I want to manage several clients with separate access so that their data stays isolated.

Requirement: Company workspaces

The system SHALL organise all data (CFDI, movements, links, jobs, audit entries) by company, and every request SHALL operate in exactly one company chosen among those the user can access.

Scenario: Switch company

- **WHEN** a user with access to two companies switches from one to the other
- **THEN** all lists and summaries show only the selected company's data

Requirement: Isolation enforced in the database

The system SHALL enforce company isolation in the data store itself, so that a defect in application code that omits a company filter cannot return another company's rows.

Scenario: Missing filter in a query

- **WHEN** a query runs in the context of company A without any company condition
- **THEN** it returns only rows of company A

Scenario: Context absent

- **WHEN** a query runs without any company context
- **THEN** it returns no rows and the attempt is logged

Requirement: Roles

The system SHALL support at least these roles per company: administrator (manage users and settings), accountant (import, review, decide) and viewer (read-only, including the audit log).

Scenario: Viewer cannot decide

- **WHEN** a viewer attempts to confirm a suggested link
- **THEN** the request is refused and nothing changes

Scenario: Accountant cannot manage users

- **WHEN** an accountant attempts to add a user to the company
- **THEN** the request is refused

Requirement: Access for several companies

The system SHALL allow one user to hold different roles in different companies.

Scenario: Different roles

- **WHEN** a user is accountant in company A and viewer in company B
- **THEN** the user can decide on links in A and only read in B

Requirement: Authenticated access only

The system SHALL require authentication for every endpoint except health checks and the API description, and SHALL store credentials only as salted, slow hashes.

Scenario: Anonymous request

- **WHEN** a request without valid credentials calls a data endpoint
- **THEN** the system refuses it without revealing whether the company exists

Requirement: Isolation verified by tests

The system SHALL include automated tests that attempt cross-company reads and writes through every data endpoint and fail if any succeeds.

Scenario: Cross-company attempt in the test suite

- **WHEN** the isolation tests request company B's invoice using a user of company A
- **THEN** the response is not-found or forbidden in every case

Requirement: Company identity

The system SHALL give each company an RFC supplied at creation, validated for format, that cannot be changed afterwards.

Scenario: Invalid RFC

- **WHEN** an operator creates a company with an RFC that has an invalid format
- **THEN** the system rejects the request with the reason

Requirement: Company creation by a platform operator

The system SHALL let a platform operator, a role outside any company, create a company with its first administrator, and SHALL record the action in the audit log.

Scenario: Company created

- **WHEN** an operator creates a company with RFC AAA010101AAA and an administrator email
- **THEN** the company exists, the administrator can sign in with the role administrator, and the creation is recorded

Scenario: Not an operator

- **WHEN** a company administrator tries to create another company
- **THEN** the request is refused

Requirement: Company settings

The system SHALL let a company administrator change the company settings (date tolerance in days, amount tolerance, complement due date moved to the next business day) and SHALL record each change in the audit log with the previous and new values.

Scenario: Tolerance changed

- **WHEN** an administrator changes the amount tolerance from 0.01 to 0.02
- **THEN** the new value applies to the next reconciliation run and the change is recorded

Scenario: Accountant cannot change settings

- **WHEN** an accountant tries to change a company setting
- **THEN** the request is refused

Requirement: Records are retained

The system SHALL NOT delete CFDI, statements, original files, links, decisions or audit entries through any API, and SHALL archive a company instead of deleting it.

Scenario: Delete attempt

- **WHEN** a user attempts to delete an imported CFDI
- **THEN** the operation is not available and nothing changes

3.7 SAT Status Check

sat-status-check

Purpose and user story. Checks whether a CFDI is current or cancelled with the SAT status service, through an interface that has a local stub first and the real service later, without changing reconciliation behaviour. User story: as an accountant, I want cancelled invoices detected so that I never reconcile against an invalid document.

Requirement: Status check contract

The system SHALL query status with the issuer RFC, receiver RFC, total and UUID, and SHALL interpret the response fields: code (N 601 invalid expression, N 602 not found, S success), state (for example current, cancelled, not found), cancellability, cancellation status (including in process and deadline expired) and the EFOS validation value.

Scenario: Current invoice

- **WHEN** the service answers success with state current
- **THEN** the system stores the status as current with the time of the check

Scenario: Cancelled invoice

- **WHEN** the service answers success with state cancelled
- **THEN** the system stores the status as cancelled and triggers review of its links

Scenario: Not found

- **WHEN** the service answers N 602
- **THEN** the system stores the status as not found and shows it distinctly from cancelled

Scenario: Invalid expression

- **WHEN** the service answers N 601
- **THEN** the system records an error for that CFDI, does not change its status and does not retry the same request

Requirement: EFOS warning

The system SHALL show a warning on any CFDI whose issuer's EFOS validation value is anything other than clear, and SHALL store the raw value returned.

Scenario: Flagged issuer

- **WHEN** the response carries the EFOS flag for the issuer
- **THEN** the CFDI is shown with a warning and the event is recorded in the audit log

Requirement: Local stub reproducing the contract

The system SHALL provide a local stub implementing the same request and response contract as the real service, and SHALL let tests force each outcome (current, cancelled, not found, invalid expression, EFOS flagged, timeout, server error, slow response).

Scenario: Forced timeout

- **WHEN** a test configures the stub to time out for a given UUID
- **THEN** the client treats it as an unavailable service for that check

Requirement: Switch by configuration

The system SHALL select the stub or the real service by configuration only, and the reconciliation logic SHALL be identical in both cases.

Scenario: Switch to the real service

- **WHEN** the configuration selects the real service
- **THEN** the same checks run against the real endpoint with no code change

Requirement: Record and replay

The system SHALL be able to record real responses once, locally, and replay them deterministically for tests and the demo, associating each recorded response with the anonymised record that stands for the real CFDI.

Scenario: Replay for anonymised data

- **WHEN** the status of an anonymised CFDI is checked in replay mode
- **THEN** the system returns the response recorded for its real counterpart and makes no network call

Requirement: Caching

The system SHALL cache a cancelled status permanently and a current status for a configurable period, and SHALL NOT query the service again for a cached status.

Scenario: Cached current status

- **WHEN** the same CFDI is checked twice within the cache period
- **THEN** only the first check calls the service

Requirement: Retries and graceful degradation

The system SHALL retry transient failures with increasing waits and a bounded number of attempts, SHALL never block imports or reconciliation because the service is unavailable, and SHALL mark affected CFDI as status unknown until a later check succeeds.

Scenario: Service down

- **WHEN** the service is unreachable during a reconciliation run
- **THEN** the run completes, affected invoices show status unknown, and the check is retried later

Requirement: Respectful request rate

The system SHALL limit its request rate to the service to a configurable maximum.

Scenario: Large batch

- **WHEN** thousands of CFDI need checking
- **THEN** requests are spread so the configured rate is not exceeded

Requirement: Cancellation process is stored

The system SHALL store, with each status, whether the CFDI is cancellable and its cancellation status (for example in process, request rejected, deadline expired), and SHALL keep a CFDI as current until the service reports it cancelled.

Scenario: Cancellation in process

- **WHEN** the service answers success with state current and cancellation status in process
- **THEN** the system stores status current with cancellation status in process

Scenario: Deadline expired

- **WHEN** the service answers state current with cancellation status deadline expired
- **THEN** the system keeps the CFDI as current and shows that status

Requirement: Check on demand

The system SHALL let an authorised user request a new status check for a CFDI as a background job, SHALL respect the rate limit, and SHALL answer from the cache, without calling the service, for a cancelled status.

Scenario: Unknown status rechecked

- **WHEN** a user requests a check for a CFDI with status unknown
- **THEN** a job is queued, and when the service answers the status is updated and the change recorded in the audit log

Scenario: Cancelled status cached

- **WHEN** a user requests a check for a CFDI already stored as cancelled
- **THEN** the system answers from the cache and does not call the service

3.8 Job Processing

job-processing

Purpose and user story. Runs imports, reconciliation and status checks in the background so that users are never blocked, and so that failures are retried without losing or duplicating work. User story: as an accountant, I want to start a large import and keep working so that I am not waiting on the screen.

Requirement: Asynchronous execution with visible progress

The system SHALL execute imports, reconciliation runs and status checks as background jobs, SHALL return immediately with a job identifier, and SHALL let users see each job's state (queued, running, succeeded, failed) and progress.

Scenario: Start an import

- **WHEN** a user uploads files
- **THEN** the system responds at once with a job identifier and the job's state changes from queued to running to succeeded

Requirement: Idempotent handlers

The system SHALL ensure that running the same job more than once, including after a crash or a redelivery, yields the same final data as running it once.

Scenario: Redelivered job

- **WHEN** a job is delivered twice to workers
- **THEN** its effects are applied once and the audit log shows no duplicate links or imports

Requirement: Retries with backoff

The system SHALL retry a failed job with increasing waits up to a configured number of attempts, and SHALL distinguish transient failures (retry) from permanent ones (fail at once).

Scenario: Transient failure then success

- **WHEN** a job fails on its first attempt because of a temporary error and succeeds on the second
- **THEN** its final state is succeeded and both attempts are recorded

Scenario: Permanent failure

- **WHEN** a job fails because its input is invalid
- **THEN** it is marked failed without retries and the reason is shown to the user

Requirement: Failed jobs are retained

The system SHALL move a job that exhausts its attempts to a failed-jobs list that keeps its input and error details, and SHALL let an administrator re-run it.

Scenario: Exhausted job

- **WHEN** a job reaches its maximum attempts
- **THEN** it appears in the failed-jobs list with its error history and can be re-run by an administrator

Requirement: No loss on worker failure

The system SHALL ensure that a job being processed when a worker stops is picked up again by another worker.

Scenario: Worker killed mid-job

- **WHEN** a worker process is terminated while processing a job
- **THEN** the job is processed to completion by another worker and its effects are applied once

Requirement: Scale out

The system SHALL allow several workers to process jobs concurrently without processing the same job simultaneously and without violating company isolation.

Scenario: Two workers

- **WHEN** two workers run against a queue of jobs from two companies

- **THEN** each job is processed by exactly one worker and each worker only touches the data of the job's company

Requirement: Reconciliation run on demand

The system SHALL let an authorised user start a reconciliation run for a company as a background job, SHALL return the existing job when a run for that company is already queued or running, and SHALL support the job types import CFDI, import statement, reconciliation run and status check.

Scenario: Run requested twice

- **WHEN** a user requests a reconciliation run while one is running for the same company
- **THEN** the system returns the running job and starts no second run

3.9 Test Data Tooling

test-data-tooling

Purpose and user story. Provides safe, realistic data for development, tests and the demo: real CFDI anonymised before they reach the repository, plus generated bank statements and edge cases. User story: as the project owner, I want realistic data without exposing anyone's information so that I can show the system publicly.

Requirement: Anonymise real CFDI

The system SHALL provide a tool that turns real CFDI XML files into anonymised files that stay structurally valid, by replacing UUIDs, RFCs, names, addresses, certificates, stamps and free-text fields, while preserving the relations between documents (an anonymised payment complement still references its anonymised invoice) and the monetary amounts.

Scenario: Relations preserved

- **WHEN** an income CFDI and the payment complement that references it are anonymised together
- **THEN** the anonymised complement references the anonymised invoice's new UUID and the amounts are unchanged

Scenario: Seals and certificates removed

- **WHEN** a CFDI is anonymised
- **THEN** the digital seals, certificate data and stamp are replaced, and the output is documented as structurally valid but not cryptographically valid

Scenario: Consistent replacement

- **WHEN** the same RFC appears in several files of one run
- **THEN** it is replaced by the same pseudonymous RFC everywhere

Requirement: Mapping kept outside the repository

The tool SHALL write the mapping between real and anonymised identifiers to a location outside the repository, and the repository SHALL ignore the folder holding raw files.

Scenario: Mapping location

- **WHEN** the tool finishes
- **THEN** the mapping exists only in the configured local location and no raw file is written inside the tracked tree

Requirement: Leak scan

The system SHALL provide a scan that fails when any file to be committed contains an identifier from the local mapping (UUID, RFC or name), and SHALL run it before every commit and in CI.

Scenario: Leaked RFC

- **WHEN** a file to be committed contains a real RFC present in the mapping
- **THEN** the scan fails and names the file, without printing the sensitive value

Requirement: Generate matching bank statements

The system SHALL generate bank statements in the reference format whose movements correspond to a set of CFDI, including exact payments, partial payments, a payment to several invoices, movements without complement, movements without any CFDI, duplicates and near-duplicates.

Scenario: Statement for the worked example

- **WHEN** the generator runs on the worked example's invoice and payment complement
- **THEN** it produces the two credit movements of 5,800.00 dated 2026-10-10 and 2026-10-20 with their descriptions

Requirement: Edge-case catalogue

The system SHALL generate CFDI and statements for these cases: cancelled invoice, cancellation in process, credit note, replacement CFDI (relation 04), invoice with discount, invoice in a foreign currency paid in pesos, PPD invoice with several complements, complement with inconsistent balance, overpaid invoice, global invoice, payment received without withholding, movement amount differing by cents, and malformed or hostile files (external entity, oversized, wrong version).

Scenario: Hostile file present

- **WHEN** the catalogue is generated
- **THEN** it includes a file with an external entity declaration and a file larger than the upload limit

Requirement: Synthetic sample from a real shape

The repository SHALL include at least one synthetic CFDI sample modelled on a real invoice shape (PPD, transferred tax and retention) containing only invented identifiers, in samples/.

Scenario: Sample contains no real data

- **WHEN** the leak scan runs over samples/
- **THEN** it finds no identifier from the local mapping

Requirement: Deterministic generation

The system SHALL produce identical output for the same seed and inputs.

Scenario: Same seed twice

- **WHEN** the generator runs twice with the same seed
- **THEN** both outputs are byte-identical

Requirement: Demo seed

The system SHALL provide a command that loads a ready-made demo dataset into a demo company, covering every state (reconciled, suggested, pending) and a cancelled invoice.

Scenario: Seed demo

- **WHEN** the demo seed runs on an empty installation
- **THEN** the demo company shows reconciled, suggested and pending items and one cancelled invoice

Requirement: Anonymisation keeps the company direction

The system SHALL replace the company's RFC consistently, so that a CFDI that was issued by the company stays issued and one that was received stays received after anonymisation.

Scenario: Direction kept

- **WHEN** an invoice issued by the company and an invoice received by the company are anonymised together
- **THEN** both keep their direction relative to the anonymised company RFC

4. Design and non-functional decisions

4.1 Context

Greenfield repository, one developer, 1–2 weeks, runs locally with Docker Compose and is shown through a recorded walkthrough (see proposal.md). The audience is reviewers for senior backend, cloud and architecture roles, so every significant choice below also becomes a short ADR with the alternatives that were rejected. Choices marked (my call) were made by the author of this document, not confirmed by the project owner, and can be changed before implementation starts.

4.2 Goals and non-goals

Goals:

- A thin vertical slice that is correct, tested and explainable: ingest → match → review → audit, with tenant isolation, idempotent asynchronous processing and a swappable SAT client.
- Few moving parts, so the whole system starts with one command and fits in a reviewer's head.
- Decisions that show trade-off reasoning, including a documented (not deployed) target cloud design.

Non-Goals:

- Public-cloud deployment, Kubernetes, formal load testing, multi-region or high-availability setups (covered only by an ADR).
- Microservices, event sourcing, machine-learning matching.
- A polished front end.

4.3 Decisions

D1. Modular monolith with a separate worker process (my call)

One codebase organised by module (ingestion, matching, review, audit, tenancy, sat, jobs) producing two entry points: the HTTP API and the worker. Modules talk through explicit interfaces, never through each other's tables.

- *Why:* one developer and two weeks; boundaries are still visible and testable; the worker scales out independently, which is what the job-processing spec needs.
- *Alternatives:* microservices (operational cost with no benefit at this size); single process with in-process jobs (cannot demonstrate worker failure and scale-out).

D2. C# on .NET 10 with ASP.NET Core, PostgreSQL (owner's choice; replaces the brief's TypeScript / NestJS proposal)

- *Why:* the project owner chose C# to build on and to learn; decimal gives exact money, ASP.NET Core and the generic host give the API and the worker from one codebase, and OpenTelemetry support is first class. PostgreSQL gives row-level security, exact NUMERIC money, transactional queues and strong constraints. Stack details are in ADR-002.
- *Alternatives:* TypeScript with NestJS (the brief's proposal; no native decimal, fewer CFDI libraries); Java with Spring Boot (strong enterprise signal, heavier for this scope); Go (more code for the rules engine); PHP, Ruby and Elixir (explored for learning, weaker fit with a 1-2 week scope); document stores (weak for N:M links and integrity).

D3. Tenant isolation with PostgreSQL row-level security

Every tenant table carries company_id. The application connects with a role that cannot bypass row- level security; each request or job starts a transaction that sets the company context, and policies filter on it. No context means no rows. Isolation tests run through every endpoint.

- *Why:* satisfies the requirement that a forgotten filter cannot leak data; strongest signal for the portfolio.
- *Alternatives:* schema per company (migrations and connection handling multiply); database per company (heavy); application-level filters only (does not meet the spec).

D4. Queue on PostgreSQL (my call; open question 6 of the BRS was unanswered)

Jobs live in a PostgreSQL table claimed with FOR UPDATE SKIP LOCKED, written in the same transaction as the domain change that created them. Visibility timeouts return jobs of dead workers to the queue; attempts, backoff and a failed-jobs list are columns and states, not extra infrastructure.

- *Why:* no new infrastructure; transactional enqueue removes a whole class of lost-job bugs; adequate for the demo's volume.
- *Alternatives:* Redis-based queue (extra service, no transactional enqueue); RabbitMQ or SQS (stronger decoupling, more to run). The ADR describes when to move to SQS or a broker in a cloud deployment.

D5. Data model

Diagrams: docs/diagrams/database/database-erd.png (tables and relationships) and database-states.png (state rules), shown in section 4.7. Core tables: company (RFC, settings), app_user, membership(role), bank_account, bank_movement (optional reference and classification), import_file (original files with hash), cfdi (UUID, type, direction, RFCs, issue time as declared, stamp time, currency, subtotal, discount, taxes, retentions, total, payment method, global flag, SAT status), cfdi_relation (SAT relation code between two CFDI), cfdi_payment (one row per payment of a complement: date, form, currency, exchange rate, amount, operation number), cfdi_payment_doc (one row per related invoice: installment, previous balance, paid, remaining, currency), link (movement, invoice, optional payment CFDI, amount applied, movement amount and rate, state, origin, criteria, confidence, decision user and time), job, sat_status_cache (status, cancellability, cancellation status, EFOS value, expiry) and audit_entry.

- Uniqueness gives idempotency: (company_id, uuid) for CFDI, a natural-key hash of account, date, description, amount and balance for movements, and one link per (movement, invoice) pair so that a rejection is remembered.
- Item states (reconciled, suggested, pending) are derived from links and amounts, not stored, so they cannot drift from the links.
- Money is NUMERIC(18,2) in storage and integer minor units in code; never floating point.
- Dates declared in a CFDI (issue time, payment date) are stored as declared, in local time without a time zone.

D6. Matching as an explainable rule pipeline

Stage 1, deterministic rules (payment complement references the invoice, amount equal, date within tolerance) produce reconciled links. Stage 2, additive heuristics (amount equal or within a few cents, date proximity, RFC found in the description, invoice number or UUID fragment in the reference text, a tracking key equal to the complement operation number, an amount equal to the total before retentions) produce suggested links with the list of criteria that fired and a confidence level (high, medium, low) derived from the points. Stage 3 evaluates partial payments and N:M allocation. User decisions are stored and take precedence; reruns are idempotent.

- *Why:* every result can be explained to an accountant and an auditor (anti-goal in the BRS); rules are unit and property testable.
- *Alternatives:* one opaque numeric score (hard to explain); learned model (no data, not explainable).

D7. Append-only, hash-chained audit log

audit_entry rows are inserted by the application role only (no update or delete privilege, plus a trigger that rejects both). Each entry stores the hash of its content and of the previous entry of the same company; a verification operation walks the chain. The entry is written in the same transaction as the action it records.

- *Alternatives:* external log store (loses atomicity with the action); event sourcing (too heavy for the slice).

D8. SAT client as a port with three adapters

A single interface with adapters for the local stub, the real SOAP service and record/replay. The stub runs as its own small container that speaks the real SOAP contract and exposes a control interface to force outcomes. Replay reads responses recorded locally and keyed to the anonymised records. Caching, retry with

backoff, rate limiting and the "status unknown" degradation sit in the port's shared layer, so all adapters behave the same.

- *Why:* phase 1 needs no account; switching to the real service is configuration only (BR-31).

D9. Safe XML handling

CFDI are parsed with a parser configured with DTD and external entity processing disabled, size and count limits at the upload boundary, and tests using the hostile files produced by the test-data tooling.

D10. Observability

Structured JSON logs with request and job identifiers; OpenTelemetry traces across API, queue and worker so one import can be followed end to end; metrics for job states, queue depth and SAT checks. Compose includes a collector and a trace and metrics viewer.

D11. API and UI

REST API with a generated OpenAPI document and versioned paths; token-based authentication with slow password hashing. A minimal single-page UI (review queue, invoice summary, overview) consumes only that API; if time runs short the API with its interactive documentation is the demonstration surface.

Endpoint catalogue (paths are proposals, frozen in the OpenAPI document of task 8.4; no DELETE exists for imported data):

- Public: GET /health, GET /openapi.json.
- Authentication: POST /auth/tokens.
- Platform operator: POST /companies (RFC and first administrator).
- Administrator: GET and PATCH /companies/{id}/settings, /users, /memberships, GET /jobs?status=failed, POST /jobs/{id}/retry.
- Accountant: GET /banks, POST /companies/{id}/cfdi-uploads, POST /companies/{id}/statements, GET /jobs/{id}, POST /companies/{id}/reconciliation-runs, GET /review-queue, POST /links/{id}/decisions (confirm, reject, change), PATCH /movements/{id} (classify), POST /cfdi/{uuid}/sat-check.
- Any role, read only: GET /cfdi (filters: type, direction, complement status), GET /cfdi/{uuid}, GET /cfdi/{uuid}/summary, GET /cfdi/{uuid}/xml, GET /movements, GET /overview, GET /audit-entries, GET /audit/verification.

D12. Testing and CI (my call on the CI host: GitHub Actions)

Unit tests; property-based tests for matching invariants (re-running changes nothing, sums are conserved, a rejected link never reappears); integration tests against a real PostgreSQL started by the test run; cross-company isolation tests on every endpoint; failure-injection tests (killed worker, SAT timeout); one end-to-end scenario for the worked example. CI builds, runs all tests, runs the leak scan and builds container images.

D13. Target cloud design is documentation only

An ADR describes how the same services would run on a managed cloud (container service, managed PostgreSQL, SQS or equivalent, secrets manager, telemetry backends), what each change would cost in complexity and money, and why it was not deployed for this project.

D14. Fiscal rules are applied to declared values

The system validates the arithmetic of the declared header amounts (total equals subtotal minus discount plus taxes minus retentions) and the balance chain of payment complements, but never recomputes taxes from rates and never validates retention rates, because rounding rules and rates change. A CFDI must involve the company's RFC; its direction (issued or received) is derived from it. Only issued income CFDI are matched in this version.

- *Why:* the SAT validates the stamped document; the platform reconciles what was declared and does not re-audit it.
- *Alternatives:* recomputing taxes (fragile around rounding); trusting totals blindly (accepts corrupt files).

D15. Retention of originals

Original XML and statement files are kept unchanged with their hash for a configurable period of at least five years; there is no delete endpoint for imported data and companies are archived, not deleted. Accounting records must be kept at least five years under the tax code (ten under the commercial code), so the safe default is ten.

- *Why:* the XML is the legal document and the statement is accounting evidence.

D16. Deadlines and cancellation are information, not enforcement

The complement due date is the fifth calendar day of the month after the payment, with a company setting to move it to the next business day (off by default). Cancellation is a process (in process, request rejected, deadline expired) and a CFDI stays current until the service says it is cancelled. The platform does not issue or cancel invoices, so it does not enforce the cancellation deadline.

4.4 Risks and trade-offs

- [Two weeks is tight for nine capabilities] → build the vertical slice first (ingest, match, review, audit); tenancy and job processing are built in from the start because they are hard to retrofit; SAT real adapter, UI polish and stretch goals go last.
- [PostgreSQL queue does not scale like a broker] → acceptable for the demo; the ADR states the limit and the migration path.
- [Row-level security adds per-transaction overhead and a risk of forgetting the context] → one transaction wrapper used by API and worker, and tests that fail when context is absent.
- [Heuristic matching may suggest wrong links] → only deterministic rules confirm automatically; everything else needs a human; reasons are always shown.
- [No real bank sample available] → the first parser targets a documented reference format; the parser interface isolates bank specifics, so a real bank adds one parser and changes nothing else.
- [Real SAT behaviour for payment CFDI is untested] → the stub follows the documented contract; the real adapter is phase 2 and its first task is a test with one real payment CFDI.
- [Anonymisation could leak or break SAT correlation] → mapping outside the repository, leak scan on every commit, and record/replay keyed to the anonymised records.
- [Several fiscal rules were verified only in secondary sources] → each one is isolated in a setting or a non-blocking warning, and the points to confirm are listed under open questions.
- [Uploading a CFDI to the wrong company] → rejected when neither RFC is the company's.

4.5 Open questions

- Which real bank's CSV is added after the reference format, once a sample is available (an extra parser only; no change to specs or approach).
- Exact tolerance defaults for date proximity and amount differences (tuned with the generated data).
- Choice of UI framework if the single-page UI is kept (does not affect the specs).
- Cancellation deadline: reported as the month of the annual return of the issue year (from 2026); informational only, to confirm with an accountant.
- Exchange rate in payment complements: an accountant's informal answer is the rate of the payment date published in the DOF; the platform stores the declared rate, to confirm.
- Cash payments: the 2,000.00 limit and the fuel rule (cash fuel is never deductible) come from secondary sources; only the limit is implemented as a warning.
- Whether the complement due date moves to the next business day when the fifth is not one: off by default, to confirm.
- Local taxes: supported in the total formula; confirm whether real files need them.
- Matching received invoices with debit movements: deferred, the model keeps the direction.

4.6 Architecture views

Figure 1 shows the running containers and how they talk to each other. Figure 2 follows one upload through the system, from the API to the worker, the SAT client and the review queue. Both are drawn from decisions D1 to D13 and are checked against the running system in task 11.2.

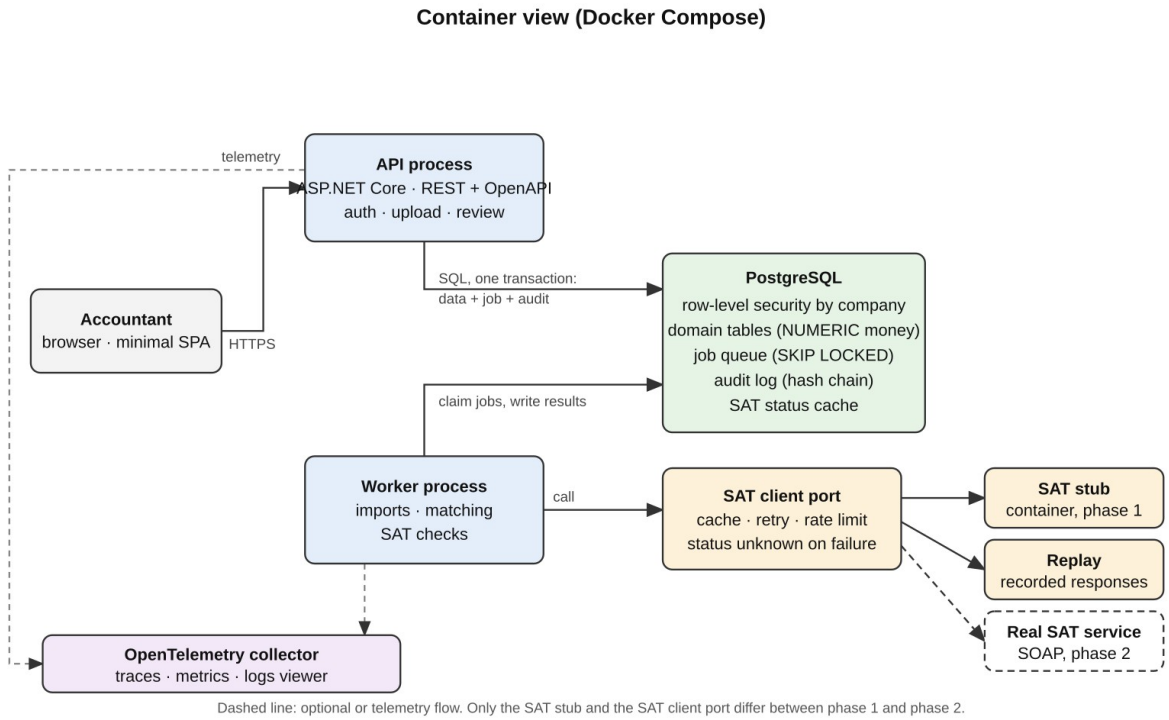


Figure 1. Container view (Docker Compose).

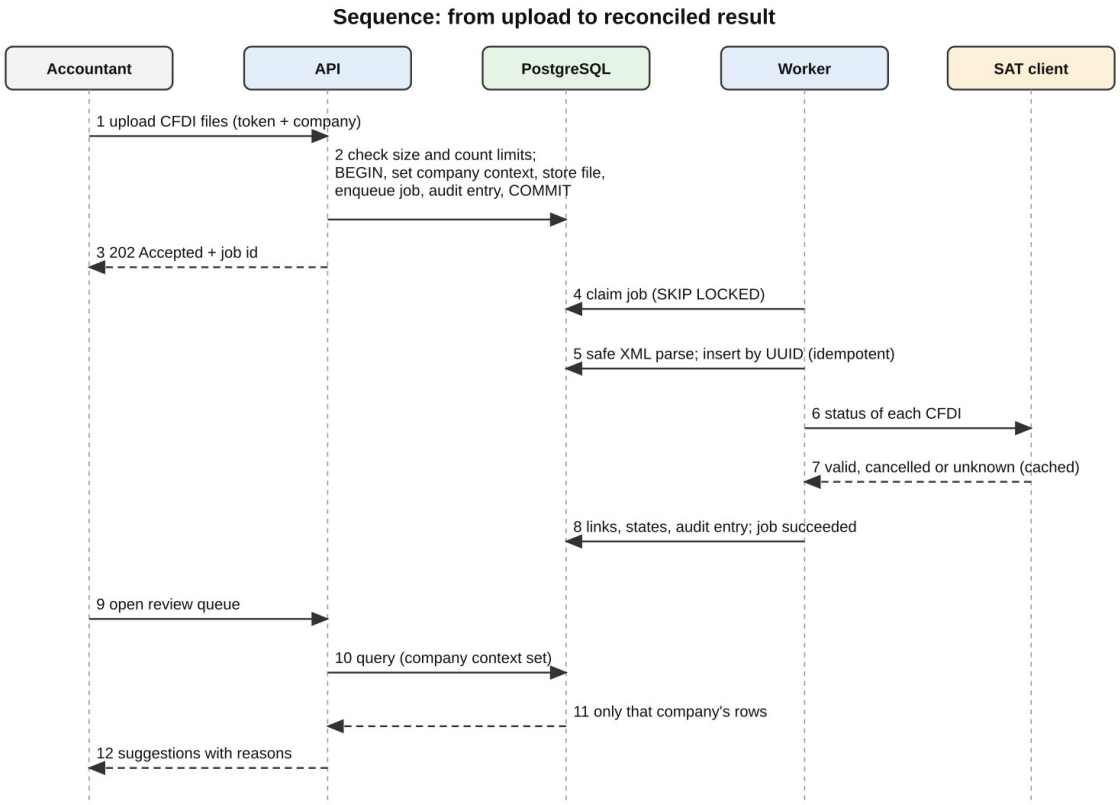


Figure 2. Sequence from upload to reconciled result.

4.7 Data model views

Figure 3 shows the tables and relationships; Figure 4 shows how link and job states change. Columns marked with * go beyond the first design, and dashed boxes are tables added by the fiscal-rule review.

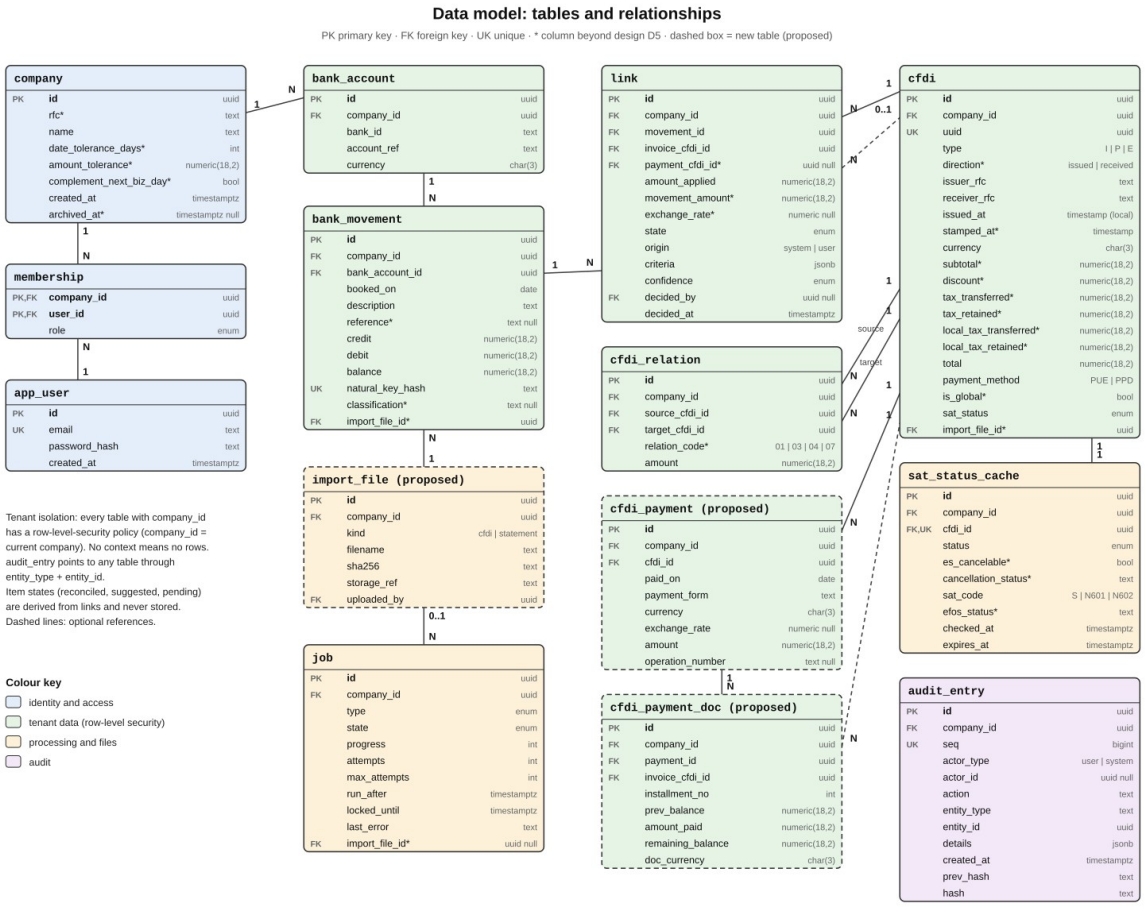


Figure 3. Data model: tables and relationships.

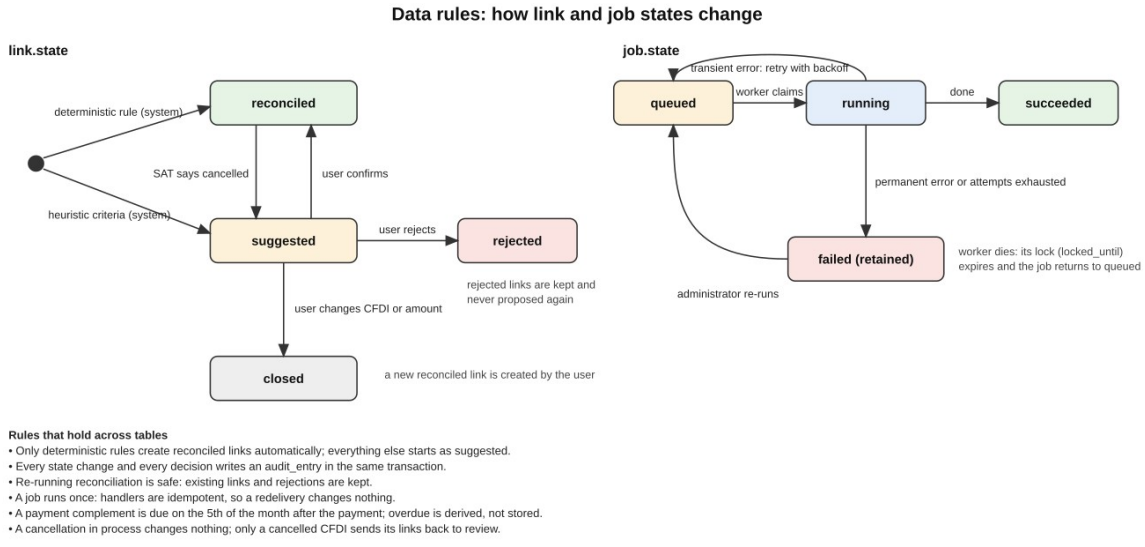


Figure 4. Data rules: link and job states.

4.8 Role sequences

Figures 5 to 9 show, for each role, how the endpoints are used. Red arrows mark endpoints added in the fiscal-rule review and orange arrows a method that changed.

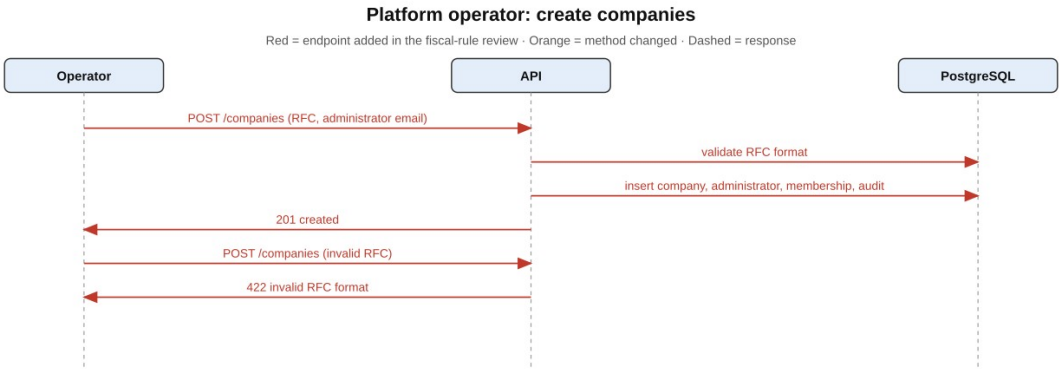


Figure 5. Platform operator.

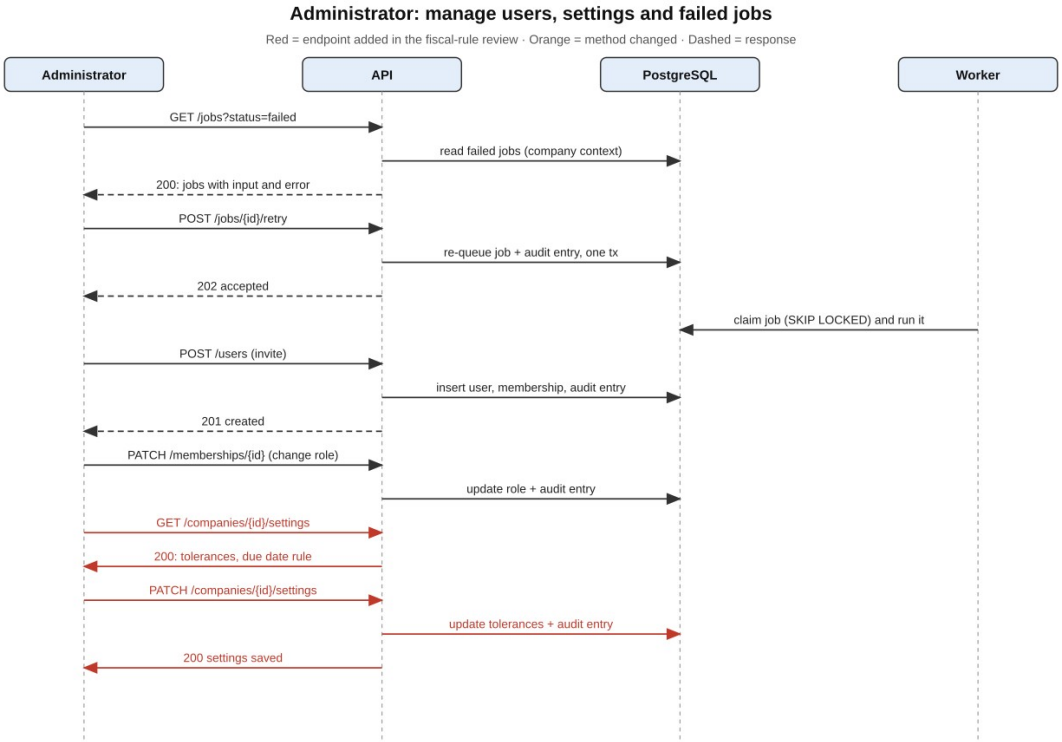


Figure 6. Administrator.



Figure 7. Accountant.

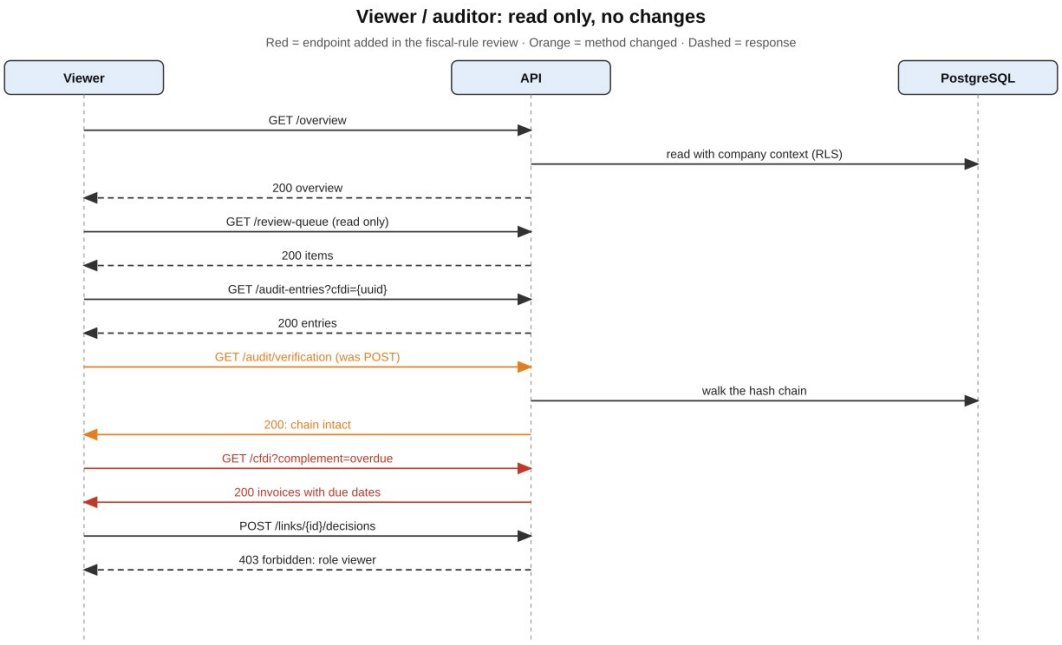


Figure 8. Viewer / auditor.

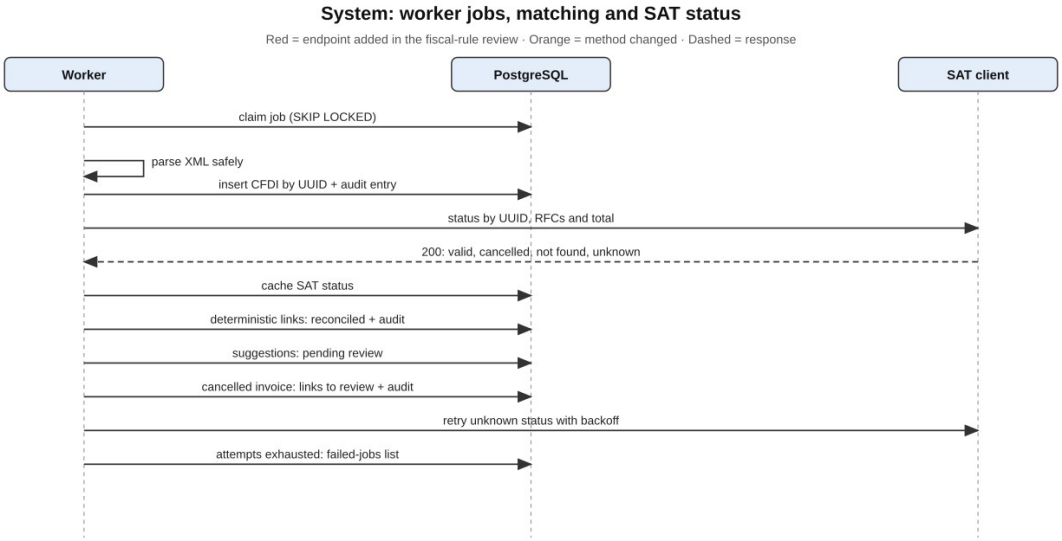


Figure 9. System (worker).

5. Implementation plan (summary of tasks)

Each task states how its completion is verified. Day ranges are the original estimate for 10 working days; the tasks added by the fiscal-rule review (2.7 to 2.9, 3.4c to 3.4f, 4.6, 6.9 to 6.15, 7.3b, 7.3c, 8.3b to 8.3d and 11.1b) add an estimated two to three days, and tasks marked stretch can be dropped if time is short.

1. Foundations (day 1)

- 1.1 Create the repository structure (API, worker, shared modules, tools, docs) and verify README lists how to build and run it
- 1.2 Add Docker Compose with PostgreSQL, API, worker and SAT stub placeholders and verify one command starts them and health checks pass
- 1.3 Add CI that builds, lints and runs tests and produces container images, and verify a pushed commit goes green
- 1.4 Write ADR-001 (modular monolith with separate worker) and ADR-002 (stack choice) and verify both list the rejected alternatives
- 1.5 Add database migrations tooling and the first migration (company, user, membership) and verify migrate up and down succeed on an empty database

2. Tenancy and access (days 1–2)

- 2.1 Implement the transaction wrapper that sets the company context for API requests and jobs, and verify a query without context returns no rows
- 2.2 Enable row-level security with policies on every tenant table, using an application role without bypass, and verify a query without a company filter returns only the context company's rows
- 2.3 Implement authentication (slow salted password hashing, token issuance) and verify an anonymous request to a data endpoint is refused
- 2.4 Implement roles (administrator, accountant, viewer) and per-company memberships, and verify the role scenarios of the tenancy spec pass
- 2.5 Write ADR-003 (tenant isolation with row-level security) and verify it compares schema-per- company and database-per-company
- 2.6 Add the cross-company isolation test suite over every data endpoint and verify it fails when a policy is deliberately removed
- 2.7 Implement company creation by a platform operator with a validated, immutable RFC, and verify the operator and non-operator scenarios
- 2.8 Implement company settings changes by administrators with audit of old and new values, and verify an accountant is refused
- 2.9 Block deletion of imported data and archive companies instead, and verify a delete attempt is unavailable

3. Domain model and ingestion (days 2–4)

- 3.1 Document the reference CSV format (columns, date and number conventions, reference tokens, sample lines) and verify the sample lines parse in task 3.6
- 3.2 Create migrations for company (RFC, settings), import_file, cfdi, cfdi_relation, cfdi_payment, cfdi_payment_doc, bank_account, bank_movement and verify uniqueness constraints reject duplicates
- 3.3 Implement safe CFDI 4.0 XML parsing (DTD and external entities disabled, size and count limits) and verify the hostile-file tests pass
- 3.4 Implement CFDI import for income, payment complement and credit note with per-file report, and verify the cfdi-ingestion scenarios pass
- 3.4b Parse discount, taxes, retentions and local taxes and verify `samples/synthetic-income-ppd-retention.xml` imports with total 1,939.27, a discount case imports, and an inconsistent total or an excessive discount is rejected

- 3.4c Reject a CFDI that does not involve the company RFC, record issued or received direction, and verify both scenarios
- 3.4d Store payments and related invoice lines of complement 2.0, flag inconsistent balances and reject other versions, and verify the three complement scenarios pass
- 3.4e Store relation codes (01, 03, 04, 07), the global-invoice flag and dates as declared, and verify the substitution, global and midnight scenarios pass
- 3.4f Keep original files with their hash and no delete path, and verify the original is returned byte for byte
- 3.5 Implement idempotent CFDI import by UUID per company and verify uploading the same file twice creates one record
- 3.6 Implement the bank parser interface, the reference-format parser and reference extraction, and verify valid, malformed, unknown-bank and tracking-key scenarios pass
- 3.7 Implement idempotent movement import with overlapping statements and verify the overlap and identical-lines scenarios pass
- 3.8 Verify money handling is exact by adding a test summing 0.10 and 0.20 to 0.30 in storage and code

4. Jobs (days 3–4)

- 4.1 Implement the PostgreSQL job queue with transactional enqueue and SKIP LOCKED claiming, and verify two workers never process the same job
- 4.2 Implement attempts, backoff, permanent versus transient failure, and the failed-jobs list with re-run, and verify the retry scenarios pass
- 4.3 Implement visibility timeout recovery and verify a worker killed mid-job leads to completion by another worker with effects applied once
- 4.4 Move CFDI and statement imports onto jobs with progress and verify an upload returns a job identifier immediately and reaches succeeded
- 4.5 Write ADR-004 (queue on PostgreSQL) with the migration path to a broker and verify it states the volume limit
- 4.6 Implement the reconciliation run job on demand with the four job types, and verify a second request returns the running job

5. Audit log (days 4–5)

- 5.1 Create audit_entry with insert-only privileges and a trigger that rejects update and delete, and verify an update attempt fails and is itself recorded
- 5.2 Implement hash chaining per company and the verification operation, and verify an altered row is reported at the correct position
- 5.3 Record audit entries inside the same transaction as imports, links, decisions and role changes, and verify a rolled-back action leaves no entry
- 5.4 Implement filtered read access for viewers and auditors and verify company isolation of the log

6. Matching engine (days 4–6)

- 6.1 Implement the link model and derived item states and verify the partial and N:M scenarios pass
- 6.2 Implement deterministic rules (payment complement, amount, date tolerance) and verify the worked example's first movement becomes reconciled
- 6.2b Use the net total (after discount and retentions) as payable amount and verify a movement of 1,939.27 fully covers the synthetic sample invoice

- 6.3 Implement heuristics with criteria, confidence and explanation, and verify the second movement becomes suggested with its reasons
- 6.4 Implement credit-note handling and verify the credit-note scenario shows nothing pending
- 6.5 Implement classification of movements without CFDI and verify they leave the pending list
- 6.6 Make reconciliation idempotent and decision-preserving, and verify the rerun and rejected- link scenarios pass
- 6.7 Add property-based tests (rerun changes nothing, sums conserved, rejected link never reappears) and verify they run in CI
- 6.8 Write ADR-005 (explainable rule pipeline) and verify it compares a single score and a learned model
- 6.9 Match only credit movements with issued income CFDI and verify a debit movement is not proposed
- 6.10 Add the gross-payment heuristic and the reference-equals-operation-number criterion, and verify both reasons appear
- 6.11 (stretch if time is short) Record invoice-currency amount, movement amount and declared rate on links, and verify the dollar-invoice scenario
- 6.12 Compute the complement due date and the overdue state with the weekend setting, and verify both scenarios
- 6.13 Flag overpaid invoices and verify the overpayment scenario
- 6.14 Keep global invoices out of RFC matching and verify they show pending with their reason
- 6.15 Cascade cancellation of invoices and complements to links, handle cancellation in process, and propose replacements (relation 04), and verify the four cancellation scenarios

7. SAT status check (days 5–6, real adapter in phase 2)

- 7.1 Define the SAT client interface with the response model and verify unit tests cover all codes and states
- 7.2 Implement the SAT stub service speaking the real SOAP contract with a control interface to force outcomes, and verify each outcome including timeout can be forced
- 7.3 Implement caching, retries with backoff, rate limiting and the unknown-status degradation, and verify the service-down scenario completes a reconciliation run
- 7.3b Store cancellability, cancellation status and the raw EFOS value, and verify the in-process and deadline-expired scenarios
- 7.3c Implement the on-demand status check job and verify the unknown and cached-cancelled scenarios
- 7.4 Connect cancelled status to matching and audit, and verify a cancelled invoice's links move to review with an audit entry
- 7.5 Implement record and replay keyed to anonymised records and verify replay makes no network call
- 7.6 Phase 2: implement the real SOAP adapter selected by configuration, test it with one real payment CFDI, and record whether the service answers for payment CFDI in the BRS open question

8. Review, summaries and API (days 6–8)

- 8.1 Implement the review queue endpoint with ordering and reasons and verify the queue scenario passes
- 8.2 Implement confirm, reject and change decisions with audit and verify the three decision scenarios pass
- 8.3 Implement invoice summary and company overview endpoints and verify the worked- example summary shows 50% paid and 5,800.00 pending

- 8.3b Add complement status to the summary, the complement filter and the overview alerts, and verify the missing, overdue and alert-count scenarios
- 8.3c Implement the supported-banks list, the movement classification, the settings and the original-file endpoints from the design catalogue, and verify each is in the OpenAPI document
- 8.3d (stretch if time is short) Show the cash payment warning above 2,000.00 and verify the state is unchanged
- 8.4 Generate the versioned OpenAPI document with the disclaimer and verify it covers every endpoint
- 8.5 Build the minimal UI (review queue, invoice summary, overview) using only the API, and verify the worked example can be completed through it

9. Test data tooling (days 3–7, in parallel)

- 9.1 Implement the generator for CFDI and matching statements with a seed, and verify identical seeds give identical output
- 9.2 Implement the edge-case catalogue including the fiscal cases (discount, foreign currency, overdue complement, global invoice, cancellation in process) and hostile files, and verify each case is produced
- 9.3 Implement the demo seed and verify an empty installation shows reconciled, suggested, pending and cancelled items
- 9.4 Implement the anonymiser preserving relations, amounts and company direction and verify the relation, consistent-replacement and direction scenarios pass
- 9.5 Implement the leak scan and wire it to pre-commit and CI, and verify it fails on a planted identifier without printing it

10. Observability and operations (days 8–9)

- 10.1 Add structured logs with request and job identifiers and verify one import can be followed in the logs
- 10.2 Add traces across API, queue and worker with a viewer in Compose and verify one import appears as a single trace
- 10.3 Add metrics for job states, queue depth and SAT checks and verify they appear in the viewer
- 10.4 Write the failure-injection tests (killed worker, SAT timeout) and verify they run in CI
- 10.5 Write the short runbook for common failures and verify each entry names a symptom, a check and a fix

11. Delivery for reviewers (days 9–10)

- 11.1 Write ADR-006 (target cloud design, not deployed) and verify it lists services, costs and why it was not deployed
- 11.1b Write ADR-007 (fiscal rules applied to declared values: scope, settings, sources and points still to confirm) and verify it lists the open questions
- 11.2 Draw the architecture views (context, containers, deployment) and verify they match the running system
- 11.3 Write the README with the problem, the worked example, the 10-minute tour and the one-command run, and verify a fresh clone follows it successfully
- 11.4 Record the walkthrough and verify it shows import, matching, review, audit verification and a trace
- 11.5 Run the whole test suite, the leak scan and the Compose start from a clean checkout and verify all pass